4.5 Algorithm of PrismSpace Multi-Agent System

The PrismSpace algorithm orchestrates multiple specialized AI agents into a coordinated swarm: interpreting user objectives, routing tasks to the optimal LLM provider, dispatching specialized agents, equipping them with tools, and verifying safety and accuracy. The workflow operates in three distinct phases:

- **Phase I: Offline Swarm Graph Representation** (Steps 1–5: Data prep, swarm graph, GAT routing)
- **Phase II: Single-Stage Preference Alignment** (Step 6: Safety and quality tuning via ORPO)
- **Phase III: Online Runtime Inference & Swarm Execution** (Steps 7–9: Live execution, evaluation, telemetry)

Inputs (What Goes In):

- **User Objective ($\mathcal{T}_{\text{user}}$):** Natural language user prompt, operational scope, and execution constraints.
- **Curated Training Corpora ($\mathcal{D}$):** Multi-source benchmark datasets containing prompt traces, intent categories, safety evaluations (WildGuard), and multi-agent trajectories (RouterBench, LMSYS Arena, CMU Agent Benchmark).
- **Preference Alignment Pairs ($\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$):** Prompt x paired with preferred response y_w and disfavored response y_l.
- **Global Agent Swarm Registry ($\mathcal{G}_{\text{global}} = (V, E)$):** Roster of specialized AI agents (Supervisor, Coding, Research, Validator, Planner) and Model Context Protocol (MCP) tool endpoints.

Outputs (What Comes Out):

- **Task Intent & Best Provider:** Predicted task category ($\hat{y}_{\text{intent}}$) and optimal LLM provider ($\hat{y}_{\text{provider}}$, e.g., Groq for rapid response, OpenAI/Anthropic for complex reasoning).
- **Swarm Dispatch Vector ($\hat{\mathbf{y}}_{\text{agents}}$):** Binary activation checklist (1 = active, 0 = idle) for each agent in V.
- **Execution Plan ($\mathcal{G}_{\text{active}}$):** Directed Acyclic Graph (DAG) outlining task steps with predicted latency ($\hat{\ell}$) and cost ($\hat{c}$).
- **Tuned Model Adapters ($\Delta\mathbf{W}$):** Lightweight LoRA weights ensuring helpfulness and safety compliance.
- **Evaluation Metrics:** Quantitative benchmarks including Accuracy, F1-Score, ROC-AUC, and ECE calibration.

Step 1: Data collection and Preprocessing

1. Import and unify raw records across multi-source benchmark datasets ($\mathcal{D}$), mapping heterogeneous prompt schemas, classification labels, and tool invocation logs into a structured, unified schema.
2. Clean and sanitize the textual dataset by stripping malformed Unicode sequences, removing redundant whitespace tokens, trimming uninformative control characters, and filtering duplicate prompt-response pairs.
3. Perform categorical encoding across discrete metadata attributes (agent role definitions, tool protocol capabilities, provider endpoints) into dense numerical vectors.
4. Standardize continuous numerical features (token length, baseline query complexity, empirical latency traces) using z-score normalization:

$$z = \frac{x - \mu}{\sigma}$$

where:

- z: Standardized continuous feature score with zero mean ($\mu = 0$) and unit variance ($\sigma = 1$), eliminating scale disparities across heterogeneous features.
- x: Raw continuous feature measurement (e.g., prompt token length, query complexity score, empirical execution latency).
- μ: Empirical arithmetic mean of the feature calculated across the training set $\mathcal{D}_{\text{train}}$.
- σ: Empirical standard deviation of the feature calculated across $\mathcal{D}_{\text{train}}$, quantifying feature dispersion.

5. Partition the processed dataset via stratified random sampling into three non-overlapping subsets:
   * 80% Training set ($\mathcal{D}_{\text{train}}$) for model parameter optimization and graph embedding learning
   * 10% Validation set ($\mathcal{D}_{\text{val}}$) for hyperparameter tuning, pruning, and early stopping
   * 10% Testing set ($\mathcal{D}_{\text{test}}$) for unbiased evaluation of routing accuracy and safety alignment

Step 2: Swarm Graph Construction

1. Define the node set (V): represent each specialized AI agent persona (Supervisor, Research, Coding, Validator, Planner) and each Model Context Protocol (MCP) tool as an individual vertex $v_i \in V$.
2. Evaluate pairwise functional capability similarity: compute the Pearson correlation coefficient between node capability vectors $\mathbf{x}_i$ and $\mathbf{x}_j$:

$$s_{ij} = \frac{\mathrm{Cov}(\mathbf{x}_i, \mathbf{x}_j)}{\sigma_i \, \sigma_j}$$

   where:

   - s_{ij}: Pearson correlation similarity coefficient between nodes v_i and v_j, bounded in $[-1, 1]$.
   - $\mathbf{x}_i, \mathbf{x}_j$: Functional capability and interface feature vectors representing nodes v_i and v_j.
   - $\mathrm{Cov}(\mathbf{x}_i, \mathbf{x}_j)$: Joint covariance measuring how capability attributes of nodes v_i and v_j co-vary.
   - σ_i, σ_j: Marginal standard deviations of capability feature distributions $\mathbf{x}_i$ and $\mathbf{x}_j$.

3. Establish collaborative topological connections: generate binary adjacency matrix $\mathbf{A}$ by thresholding similarity against cutoff τ:

$$A_{ij} = \begin{cases} 1 & \text{if } s_{ij} \geq \tau \\ 0 & \text{otherwise} \end{cases}$$

   where:

   - A_{ij}: Binary connectivity entry in adjacency matrix $\mathbf{A} \in \{0, 1\}^{|V| \times |V|}$, denoting presence (1) or absence (0) of an active collaborative edge between nodes v_i and v_j.
   - s_{ij}: Pairwise Pearson correlation similarity coefficient between nodes v_i and v_j.
   - τ: Empirical correlation cutoff threshold regulating swarm topology density and pruning weak pairings.

4. Construct the global swarm topology $\mathcal{G} = (V, E)$: establish undirected collaborative communication edges E that permit message propagation exclusively between complementary agent and tool pairings.

Step 3: Graph Attention Network (GAT) Initialization

1. Initialize node representations: assign each vertex $v_i \in V$ an initial d-dimensional feature embedding vector:

$$\mathbf{h}_i = \mathbf{x}_i \in \mathbb{R}^d, \qquad \mathbf{W} \in \mathbb{R}^{d' \times d}, \qquad \mathbf{a} \in \mathbb{R}^{2d'}$$

   where:

   - $\mathbf{h}_i$: Initial feature representation vector assigned to vertex $v_i \in V$.
   - $\mathbf{x}_i$: Raw input attribute vector of vertex v_i encoding operational scope and tool interfaces.
   - d: Dimensionality of input feature space.
   - $\mathbf{W}$: Shared learnable linear projection weight matrix mapping input features to latent dimension d'.
   - d': Projected latent feature representation dimensionality.
   - $\mathbf{a}$: Learnable shared attention mechanism vector used to compute inter-node attention scores.

2. Initialize projection and attention parameters: instantiate weight projection matrix $\mathbf{W}$ and attention vector $\mathbf{a}$ via Xavier uniform initialization, ensuring stable variance across forward and backward signal propagation.
3. Configure network hyperparameters: establish learning rate η, attention head count K (specialized across task intent, model capability, swarm affinity, and safety constraints), and dropout regularization probability p.

Step 4: Attention-Based Message Passing

For each node v_i with immediate collaborative neighborhood $\mathcal{N}(i)$:

1. Compute raw attention coefficients with connected neighbors $v_j \in \mathcal{N}(i)$:

$$e_{ij} = \mathrm{LeakyReLU}\Big(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \,\|\, \mathbf{W}\mathbf{h}_j]\Big)$$

where:

- e_{ij}: Raw unnormalized attention coefficient indicating communicative significance of neighbor v_j to node v_i.
- $\mathbf{a} \in \mathbb{R}^{2d'}$: Parameterized shared attention mechanism weight vector.
- $\mathbf{a}^T$: Transpose of attention vector $\mathbf{a}$, computing inner product with concatenated node representations.
- $\mathbf{W} \in \mathbb{R}^{d' \times d}$: Shared linear feature projection matrix applied to each node representation.
- $\mathbf{h}_i, \mathbf{h}_j$: Current hidden state feature vectors of receiving node v_i and neighboring source node v_j.
- $\|$: Vector concatenation operator joining projected vectors $\mathbf{W}\mathbf{h}_i$ and $\mathbf{W}\mathbf{h}_j$ into a single $2d'$-dimensional vector.
- $\mathrm{LeakyReLU}(\cdot)$: Non-linear activation function allowing gradient flow even for negative inputs.

2. Normalize coefficients across the local neighborhood using Softmax:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

where:

- α_{ij}: Normalized attention weight representing communicative attention directed from node v_i to neighbor v_j, satisfying $\sum_{j \in \mathcal{N}(i)} \alpha_{ij} = 1$.
- e_{ij}: Raw attention coefficient computed between target node v_i and neighbor v_j.
- $\mathcal{N}(i)$: Neighborhood vertex set containing all nodes directly connected to node v_i in swarm topology $\mathcal{G}$.
- $\exp(\cdot)$: Natural exponential function guaranteeing strictly positive weights.
- $\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})$: Softmax normalization denominator summing exponentiated attention scores across all neighbors of node v_i.

3. Aggregate neighborhood feature representations (single-head aggregation):

$$\mathbf{h}'_i = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j\right)$$

where:

- $\mathbf{h}'_i$: Updated, context-aware output feature representation vector for node v_i.
- $\sigma(\cdot)$: Non-linear activation function (such as Exponential Linear Unit, ELU).
- α_{ij}: Normalized attention weight scaling the message representation of neighbor v_j.
- $\mathbf{W}$: Linear feature projection weight matrix.
- $\mathbf{h}_j$: Hidden feature vector of neighboring node v_j.
- $\mathcal{N}(i)$: Direct neighborhood set of node v_i.

4. Multi-head attention concatenation across K independent attention heads:

$$\mathbf{h}'_i = \|_{k=1}^{K} \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k \mathbf{W}^k \mathbf{h}_j\right)$$

where:

- $\mathbf{h}'_i$: Final concatenated multi-head embedding of node v_i, of dimension $K \cdot d'$.
- $\|_{k=1}^{K}$: Vector concatenation operator combining representations learned across all K independent attention heads.
- K: Total count of parallel attention heads specialized across distinct collaborative dimensions.
- α_{ij}^k: Normalized attention coefficient produced by the k-th attention head for neighbor v_j.
- $\mathbf{W}^k$: Transformation weight matrix parameterized specifically for the k-th attention head.
- $\mathbf{h}_j$: Feature vector of neighbor v_j.
- $\sigma(\cdot)$: Non-linear activation function.

Step 5: Classification and Loss Computation

1. Route updated swarm representations $\mathbf{h}'_i$ through dual specialized prediction heads:
   * *Intent & Provider Routing Head:* Computes normalized routing probabilities via Softmax:

$$\hat{\mathbf{y}}_{\text{route}} = \text{Softmax}(\mathbf{W}_{\text{route}}\mathbf{h}' + \mathbf{b}_{\text{route}})$$

   where:
   - $\hat{\mathbf{y}}_{\text{route}}$: Predicted probability distribution vector over available foundation LLM providers (e.g., Groq, OpenAI, Anthropic).
   - $\mathbf{W}_{\text{route}}$: Trainable projection weight matrix for the routing classification head.
   - $\mathbf{h}'$: Graph-level pooled feature embedding summarizing the entire swarm graph and user query.
   - $\mathbf{b}_{\text{route}}$: Trainable additive bias vector of the routing head.
   - $\text{Softmax}(\cdot)$: Normalization function converting raw logit scores into valid probabilities summing to 1.

   * *Agent Dispatch Head:* Evaluates independent activation probabilities per agent persona via element-wise Sigmoid:

$$\hat{y}_{\text{agent},i} = \text{Sigmoid}(\mathbf{W}_{\text{agent}}\mathbf{h}'_i + b_i)$$

   where:
   - $\hat{y}_{\text{agent},i}$: Predicted activation probability in $[0, 1]$ indicating whether agent persona v_i should be dispatched.
   - $\mathbf{W}_{\text{agent}}$: Trainable projection parameter vector for multi-label agent dispatching.
   - $\mathbf{h}'_i$: Contextualized feature embedding of node v_i output by the multi-head GAT layer.
   - b_i: Trainable scalar bias term for agent node v_i.
   - $\text{Sigmoid}(\cdot)$: Logistic activation function mapping real-valued logits to independent probabilities in $[0, 1]$.

2. Compute unified multi-task loss ($\mathcal{L}_{\text{class}}$) combining routing cross-entropy and agent dispatch cross-entropy:

$$\mathcal{L}_{\text{class}} = \mathcal{L}_{\text{CE}}(\mathbf{y}_{\text{route}}, \hat{\mathbf{y}}_{\text{route}}) + \lambda \sum_{v \in V} \mathcal{L}_{\text{BCE}}(y_v, \hat{y}_v)$$

   where:
   - $\mathcal{L}_{\text{class}}$: Total multi-task joint classification objective minimized during offline training.
   - $\mathcal{L}_{\text{CE}}$: Categorical Cross-Entropy loss measuring routing classification error against ground truth.
   - $\mathbf{y}_{\text{route}}$: Ground-truth one-hot target vector representing optimal provider selection.
   - $\hat{\mathbf{y}}_{\text{route}}$: Model-predicted probability vector over provider classes.
   - $\mathcal{L}_{\text{BCE}}$: Binary Cross-Entropy loss evaluating activation prediction for each candidate agent node v.
   - y_v: Ground-truth binary activation label (1 if agent v is required, 0 otherwise).
   - $\hat{y}_v$: Predicted activation probability for agent v.
   - λ: Regularization hyperparameter balancing provider routing accuracy against agent over-provisioning penalties.
   - V: Universal set of agent persona vertices in the swarm registry.

3. Optimize parameters: backpropagate error gradients and update network parameters using the AdamW optimizer with cosine learning-rate decay and gradient norm clipping.

Step 6: Single-Stage Preference Alignment (ORPO)

1. Inject Parameter-Efficient Fine-Tuning (PEFT) adapters via low-rank decomposition matrices:

$$\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{B}\mathbf{A}$$

where $\Delta \mathbf{W} \in \mathbb{R}^{d \times k}$ represents the low-rank incremental weight update adapter added to frozen base weights $\mathbf{W}_0$, $\mathbf{B} \in \mathbb{R}^{d \times r}$ is the trainable down-projection matrix (zero-initialized), $\mathbf{A} \in \mathbb{R}^{r \times k}$ is the trainable up-projection matrix (Gaussian initialized), r is the intrinsic rank hyperparameter ($r \ll \min(d, k)$), and α is the LoRA scaling constant controlling update magnitude.

2. Compute the supervised fine-tuning (SFT) objective on favored human responses (y_w):

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|y_w|} \sum_{t=1}^{|y_w|} \log P(y_{w,t} \mid x, y_{w,<t})$$

where:

- $\mathcal{L}_{\text{SFT}}$: Negative log-likelihood sequence loss over favored human responses, maintaining fluent language syntax.
- x: Input prompt sequence representing the user objective and operational context.
- y_w: Preferred (winning) demonstration token sequence of length $|y_w|$.
- $y_{w,t}$: The target ground-truth token generated at step t.
- $y_{w,<t}$: Autoregressive prefix context consisting of all tokens preceding position t ($y_{w,1}, \ldots, y_{w,t-1}$).
- $P(y_{w,t} \mid x, y_{w,<t})$: Conditional probability assigned to token $y_{w,t}$ by the language model given prompt x and preceding context tokens.

3. Calculate length-normalized sequence odds for completion sequence $y \in \{y_w, y_l\}$:

$$\text{odds}(y \mid x) = \frac{P(y \mid x)}{1 - P(y \mid x)}$$

where $\text{odds}(y \mid x)$ represents the odds ratio of generating sequence y given prompt x, $P(y \mid x)$ is the length-normalized likelihood $\left(\prod_{t=1}^{|y|} P(y_t \mid x, y_{<t})\right)^{1/|y|}$, and $1 - P(y \mid x)$ is the complementary non-generation probability.

4. Formulate the Odds Ratio (OR) preference penalty between favored (y_w) and disfavored (y_l) completions:

$$\mathcal{L}_{\text{OR}} = -\log \sigma \left(\log \left[\frac{\text{odds}(y_w \mid x)}{\text{odds}(y_l \mid x)} \right] \right)$$

where:

- $\mathcal{L}_{\text{OR}}$: Odds Ratio contrastive penalty that penalizes disfavored patterns (hallucinations, safety violations) and rewards preferred patterns.
- $\sigma(\cdot)$: Standard logistic sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$.
- $\text{odds}(y_w \mid x)$: Sequence generation odds for the preferred / winning completion y_w.
- $\text{odds}(y_l \mid x)$: Sequence generation odds for the disfavored / losing completion y_l.
- $\log \left[\frac{\text{odds}(y_w \mid x)}{\text{odds}(y_l \mid x)} \right]$: Log odds ratio measuring the log-margin difference between favored and disfavored outputs.

5. Optimize unified single-stage ORPO objective:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \beta \cdot \mathcal{L}_{\text{OR}}$$

where $\mathcal{L}_{\text{ORPO}}$ represents the composite single-stage objective minimized without external reward models, $\mathcal{L}_{\text{SFT}}$ is the supervised fine-tuning loss ensuring language fluency, $\mathcal{L}_{\text{OR}}$ is the odds ratio contrast loss penalizing disfavored generation modes, and β is the regularization weighting coefficient balancing alignment against SFT fluency.

Step 7: Real-Time Runtime Inference and Swarm Execution

1. Capture user objective: ingest natural language prompt $\mathcal{T}_{\text{user}}$ via the Next.js client interface and securely dispatch it to the FastAPI Hive Bridge orchestration gateway.
2. Synthesize query feature vector ($\mathbf{x}_{\text{query}}$): concatenate sparse TF-IDF n-gram vectors, dense semantic sentence embeddings, and session contextual metadata (token budget, latency requirements, user tool permissions).
3. Determine routing and dispatch policy: assign prompt to optimal LLM provider and activate required swarm agents:

$$\hat{y}_{\text{provider}} = \operatorname{argmax}(\hat{\mathbf{y}}_{\text{route}}), \qquad \hat{\mathbf{y}}_{\text{agents}} = \mathbb{I}(\hat{\mathbf{y}}_{\text{dispatch}} \geq \tau_{\text{dispatch}})$$

   where:

   - $\hat{y}_{\text{provider}}$: Index of selected foundation LLM provider (e.g., Groq for rapid latency, OpenAI/Anthropic for complex reasoning).
   - $\hat{\mathbf{y}}_{\text{route}}$: Posterior probability distribution vector output by the routing prediction head.
   - $\operatorname{argmax}(\cdot)$: Selection operator returning provider index with maximum routing probability.
   - $\hat{\mathbf{y}}_{\text{agents}}$: Binary dispatch checklist vector (1 = active agent, 0 = idle agent) defining the dispatched swarm.
   - $\mathbb{I}(\cdot)$: Element-wise indicator function returning 1 when condition holds and 0 otherwise.
   - $\hat{\mathbf{y}}_{\text{dispatch}}$: Continuous activation probability vector output by the agent dispatch head.
   - τ_{dispatch}: Calibrated activation decision threshold for triggering agent instantiation.
4. Formulate task DAG and execution schedule via topological sorting:

$$\pi = \operatorname{TopologicalSort}(\mathcal{G}_{\text{active}})$$

   where:

   - π: Linearly ordered execution schedule of subtask steps ensuring prerequisite tasks complete before downstream tasks begin.
   - $\mathcal{G}_{\text{active}}$: Active execution Directed Acyclic Graph (DAG) whose vertices represent subtasks and directed edges denote data dependencies.
   - $\operatorname{TopologicalSort}(\cdot)$: Graph algorithm producing a valid linear ordering where all prerequisite dependencies are satisfied.
5. Execute autonomous ReAct swarm loops: dispatch agents to iteratively reason, propose actions, and invoke Model Context Protocol (MCP) tool endpoints (Browser, Filesystem, GitHub, Python Sandbox), aggregating intermediate tool observation traces until all subtask criteria are satisfied.
6. Enforce policy safety gating: evaluate final candidate completions through the calibrated safety gate, verifying that the log-odds safety score relative to reference text meets safety threshold τ_{safety}, intercepting prompt injection, unsafe operations, or policy violations.
7. Deliver real-time response and persist session state: stream verified response tokens live to the frontend via Server-Sent Events (SSE), update real-time telemetry HUD metrics, and commit complete conversation artifacts to local browser IndexedDB storage using Dexie.js.

Step 8: Model Evaluation

1. Assess generalization performance: benchmark trained GAT routing and ORPO alignment models against holdout validation ($\mathcal{D}_{\text{val}}$) and test ($\mathcal{D}_{\text{test}}$) splits to quantify accuracy and preference adherence.
2. Compute comprehensive quantitative validation metrics:
   - *Routing Accuracy & Macro-F1:* Measure proportion of correctly classified intents and provider selections, computing harmonic mean of precision and recall across rare and frequent task classes.
   - *Discrimination Capability (ROC-AUC & AUPRC):* Measure the area under the receiver operating characteristic and precision–recall curves to assess safety gate discrimination under class imbalance.
   - *Calibration Quality (ECE):* Measure Expected Calibration Error across M probability bins:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|$$

     where:
     - ECE: Expected Calibration Error, quantifying overall alignment between predicted confidence scores and observed empirical accuracy.
     - M: Number of discrete confidence bins partitioning the prediction score interval $[0, 1]$ (e.g., $M = 10$).
     - B_m: The set of sample instances whose predicted confidence score falls into bin m with cardinality $|B_m|$.
     - N: Total number of evaluated test prediction instances across all M bins.
     - $\text{acc}(B_m)$: Empirical accuracy observed for test samples assigned to bin B_m.
     - $\text{conf}(B_m)$: Average predicted confidence probability for test samples assigned to bin B_m.
     - $|\text{acc}(B_m) - \text{conf}(B_m)|$: Absolute calibration error gap within bin B_m.
   - *Regression Precision (R^2 & MAE):* Quantify predictive fidelity of task latency ($\hat{\ell}$) and monetary token cost ($\hat{c}$):

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}, \qquad \text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

     where R^2 denotes the coefficient of determination quantifying explained variance, MAE is the mean absolute error in latency (ms) or token cost (USD), y_i is the empirical target value, $\hat{y}_i$ is the predicted metric, $\bar{y}$ is the empirical benchmark mean, and N is the total count of evaluated traces.
3. Save optimized model checkpoints, export weight artifacts, and log diagnostic performance reports.

Step 9: Visualization and Interpretation

- ▶ Generate convergence diagnostics: plot training and validation loss trajectories, ROC curves, and precision–recall frontiers to track learning progression and over-fitting boundaries.
- ▶ Inspect attention heatmaps: visualize inter-agent and agent-tool attention weight matrices across the K attention heads, identifying dominant collaboration pathways and tool utilization dependencies.
- ▶ Monitor live operational telemetry: track real-time routing hit rates, token cost allocations ($\hat{c}$), task latency profiles ($\hat{\ell}$), and safety policy pass rates on the Swarm Telemetry HUD.

Algorithm Summary

1. **Graph-Attentive Swarm Routing:** The GAT architecture models functional interactions between specialized agents and MCP tools, dynamically routing incoming tasks to the optimal LLM provider and dispatching minimal required worker agents.
2. **Single-Stage Preference Alignment:** The ORPO formulation unifies supervised fine-tuning with length-normalized odds ratio penalties, tuning model helpfulness and safety in a single stage without independent reward models.
3. **Autonomous Reactive Execution:** Runtime execution is scheduled via DAG topological sorting, iterative ReAct tool loops, rigorous odds-ratio safety gating, and live SSE streaming to browser IndexedDB storage.